

Tarefa 9: Regiões críticas nomeadas e Locks explícitos

Israel Soares de Castro Filho
Matrícula: 20250070780

1 Introdução

Este relatório apresenta os resultados de dois programas em C, cujo objetivo é realizar N inserções em listas encadeadas de forma concorrente, garantindo a integridade das estruturas de dados e evitando condições de corrida.

O primeiro programa (`duas_listas.c`) trabalha com um número fixo de duas listas e utiliza *regiões críticas nomeadas* (`#pragma omp critical(nome)`) para sincronizar o acesso a cada lista de forma independente. O segundo programa (`n_listas.c`) generaliza a solução para um número M de listas definido pelo usuário em tempo de execução, o que exige a substituição das regiões críticas nomeadas por um vetor de *locks* explícitos (`omp_lock_t`), já que os nomes de regiões críticas precisam ser conhecidos em tempo de compilação.

O objetivo deste relatório é analisar as saídas produzidas pelos dois programas para diferentes valores de N (e de M , no segundo caso), verificando a corretude das inserções e o balanceamento da distribuição aleatória entre as listas.

2 Metodologia

Ambos os programas seguem a mesma estrutura geral: dentro de uma região `#pragma omp parallel`, uma única thread, definida pela diretiva `#pragma omp single`, executa o laço `for`. Em cada iteração, a diretiva `#pragma omp task` cria uma nova tarefa, totalizando **N tarefas**. Essas tarefas são então distribuídas dinamicamente entre as threads da região paralela pela execução.

Cada tarefa sorteia, de forma independente, um número de lista e um valor inteiro a ser inserido, utilizando um gerador Xorshift32 com uma semente própria (`firstprivate`), tornando o sorteio thread-safe e dispensando sincronização nessa etapa.

- Em `duas_listas.c`, a lista sorteada é sempre `lista1` ou `lista2`, e a inserção é protegida por duas regiões críticas nomeadas distintas, permitindo que inserções em listas diferentes ocorram em paralelo.
- Em `n_listas.c`, o número de listas M é lido em tempo de execução. Um vetor de M locks (`omp_lock_t`) é criado dinamicamente, um para cada lista, e a inserção na lista de índice `idx` é protegida por `omp_set_lock(&locks[idx])` e `omp_unset_lock(&locks[idx])`.

Os dois programas foram executados para diferentes tamanhos de entrada. Para `duas_listas.c`, o número de inserções N variou entre 20 e 200.000. Para `n_listas.c`, o

número de inserções variou entre 40 e 400.000, combinado com $M \in \{4, 8, 16\}$ listas. Ao final de cada execução, o programa imprime a quantidade de elementos em cada lista, permitindo verificar se a soma dos elementos de todas as listas é igual ao número de inserções solicitado (o que indicaria ausência de perdas ou duplicações causadas por condições de corrida).

3 Resultados

3.1 Programa com duas listas

A Tabela 1 apresenta o número de elementos inseridos em cada lista, o percentual de cada lista em relação ao total, e a conferência do total inserido, para diferentes valores de N .

Tabela 1: Resultados de `duas_listas.c` para diferentes valores de N

N	Lista 1	% Lista 1	Lista 2	% Lista 2	Total confere
20	10	50,00%	10	50,00%	Sim
200	92	46,00%	108	54,00%	Sim
2.000	1.025	51,25%	975	48,75%	Sim
20.000	10.071	50,35%	9.929	49,65%	Sim
200.000	100.079	50,04%	99.921	49,96%	Sim

3.2 Programa generalizado para M listas

A Tabela 2 resume as execuções de `n_listas.c` para $M \in \{4, 8, 16\}$ listas. Em vez de listar a contagem individual de cada uma das até 16 listas, apresentam-se estatísticas de balanceamento: o valor esperado por lista (N/M , assumindo distribuição perfeitamente uniforme), a menor e a maior contagem observadas entre as listas, e o desvio percentual de cada uma em relação ao valor esperado.

Em todas as execuções, tanto de `duas_listas.c` quanto de `n_listas.c`, a soma dos elementos de todas as listas foi exatamente igual ao número N de inserções solicitado, sem nenhuma perda ou duplicação de nós.

4 Análise

Corretude. Em todas as execuções, a soma dos elementos das listas foi exatamente igual a N , tanto em `duas_listas.c` quanto em `n_listas.c` (Tabelas 1 e 2). Isso confirma que as duas estratégias de exclusão mútua utilizadas – regiões críticas nomeadas em um caso, locks explícitos no outro – protegeram corretamente o acesso concorrente às listas: nenhuma tarefa perdeu ou duplicou um nó ao inserir simultaneamente com outra.

Balanceamento entre listas. Como cada tarefa escolhe a lista de destino de forma aleatória e uniforme, o número esperado de elementos por lista é $N/2$ em `duas_listas.c` e N/M em `n_listas.c`. As oscilações em torno desse valor esperado diminuem à medida que N cresce (Lei dos Grandes Números) e aumentam à medida que M cresce para um mesmo N , já que cada lista passa a receber, em média, menos inserções e amostras menores variam proporcionalmente mais. Isso é visível comparando, por exemplo, $N = 400$ com

Tabela 2: Resultados de `n_listas.c` para diferentes valores de N e M

N	M	Esperado/lista	Mín.	Desvio mín.	Máx.	Desvio máx.
40	4	10,0	3	-70,00%	16	+60,00%
400	4	100,0	97	-3,00%	105	+5,00%
4.000	4	1.000,0	970	-3,00%	1.024	+2,40%
40.000	4	10.000,0	9.939	-0,61%	10.068	+0,68%
400.000	4	100.000,0	99.544	-0,46%	100.667	+0,67%
400	8	50,0	38	-24,00%	61	+22,00%
4.000	8	500,0	481	-3,80%	519	+3,80%
40.000	8	5.000,0	4.879	-2,42%	5.083	+1,66%
400.000	8	50.000,0	49.710	-0,58%	50.161	+0,32%
400	16	25,0	16	-36,00%	35	+40,00%
4.000	16	250,0	221	-11,60%	284	+13,60%
40.000	16	2.500,0	2.399	-4,04%	2.561	+2,44%
400.000	16	25.000,0	24.735	-1,06%	25.202	+0,81%

$M = 4$ (desvio máximo de +5%) e $N = 400$ com $M = 16$ (desvio máximo de +40%) na Tabela 2. Esses desvios são um efeito estatístico do sorteio, não um sintoma de falha de sincronização.

Por que regiões críticas nomeadas não são suficientes quando M é dinâmico. Em `duas_listas.c` o número de listas é fixo e igual a 2, conhecido já no código-fonte. Isso torna viável criar, à mão, uma região crítica nomeada para cada lista – `#pragma omp critical(lista1_critico)` e `#pragma omp critical(lista2_critico)` – de modo que inserções em listas diferentes não disputem o mesmo mutex e possam ocorrer em paralelo, enquanto inserções na mesma lista são serializadas.

O problema é que o nome que aparece dentro dos parênteses de:

```
#pragma omp critical(nome)
```

É um identificador fixado em tempo de compilação: ele precisa existir como texto no código-fonte antes mesmo de o programa rodar. Em `n_listas.c`, o número de listas M é lido de `argv` em tempo de execução, então não há como o compilador saber, ao gerar o binário, se serão necessárias 4, 8, 16 ou qualquer outra quantidade de regiões críticas – não é possível escrever "de antemão" um nome de região crítica para cada uma das M listas, pois M ainda não existe nesse momento.

Por que locks explícitos resolvem o problema. A diferença essencial é que um `omp_lock_t` não é um identificador textual do código-fonte, mas um objeto em memória que pode ser criado, indexado e manipulado em tempo de execução. Em `n_listas.c`, isso é usado da seguinte forma:

1. Um vetor de M locks é alocado dinamicamente depois que M é conhecido:

```
omp_lock_t *locks = malloc((size_t)M * sizeof(omp_lock_t));
```
2. Cada posição do vetor é inicializada em um laço `for`, algo que só faz sentido porque M é uma variável, não uma constante de compilação:

```
for (int i = 0; i < M; i++) omp_init_lock(&locks[i]);
```
3. Dentro de cada tarefa, o índice `idx` da lista sorteada é usado para indexar o vetor de locks em tempo de execução

Assim, obtém-se exatamente a mesma propriedade que as regiões críticas nomeadas forneciam em `duas_listas.c` – um mutex independente por lista, permitindo que inserções em listas diferentes ocorram verdadeiramente em paralelo, enquanto inserções na mesma lista se sincronizam entre si – só que com a quantidade de mutexes decidida em tempo de execução, e não fixada no código-fonte.

Regiões críticas nomeadas exigem que o número de "categorias" de exclusão mútua seja conhecido em tempo de compilação; um vetor de locks explícitos, por poder ser alocado e indexado dinamicamente, é a ferramenta que preserva tanto a corretude quanto o paralelismo quando esse número, M , só é definido pelo usuário na hora da execução.

5 Conclusão

Os resultados obtidos confirmam que ambas as estratégias de sincronização – regiões críticas nomeadas para um número fixo e pequeno de listas, e locks explícitos para um número M definido em tempo de execução – garantem a integridade das listas encadeadas durante inserções concorrentes, sem perda ou duplicação de elementos em nenhuma das execuções testadas. As oscilações observadas na distribuição de elementos entre as listas, mais acentuadas para valores pequenos de N e maiores de M , são explicadas pela variância estatística natural do sorteio aleatório e diminuem à medida que N cresce, sem qualquer relação com falhas de sincronização.

Do ponto de vista de projeto, a comparação reforça que regiões críticas nomeadas são adequadas apenas quando o número de categorias de exclusão mútua é conhecido em tempo de compilação; quando esse número é definido dinamicamente pelo usuário, como no caso de `n_listas.c`, o uso de locks explícitos organizados em um vetor é a alternativa que preserva tanto a corretude quanto o paralelismo entre listas distintas.

A Código-Fonte

A.1 duas_listas.c

```
1  /*
2   * duas_listas.c
3   *
4   * Compilar: gcc -fopenmp -Wall -O2 duas_listas.c -o duas_listas
5   * Executar: ./duas_listas [N]
6   */
7
8  #include <stdio.h>
9  #include <stdlib.h>
10 #include <time.h>
11 #include <omp.h>
12
13 /* -----
14  * Gerador Xorshift32 (thread-safe: cada chamador guarda seu próprio 'state')
15  * ----- */
16 static inline unsigned int xorshift32(unsigned int *state)
17 {
18     unsigned int x = *state;
19
20     x ^= x << 13;
21     x ^= x >> 17;
22     x ^= x << 5;
23
24     *state = x;
25     return x;
26 }
27
28 /* Geracao de double aleatorio em [0,1) */
29 static inline double random_double(unsigned int *state)
30 {
31     return (double)xorshift32(state) / (double)UINT_MAX;
32 }
33
34 static inline unsigned int semente_da_thread(void)
35 {
36     unsigned int base = (unsigned int)time(NULL) ^
37                         ((unsigned int)omp_get_thread_num() * 2654435761u);
38     return base | 1u;
39 }
40
41
42 typedef struct Node {
43     int data;
44     struct Node *next;
45 } Node;
46
47 /* Insere no inicio da fila -- Não éthread-safe pro si só*/
48 static void insere_lista(Node **head, int valor) {
49     Node *novo = malloc(sizeof(Node));
50     novo->data = valor;
51     novo->next = *head;
52     *head = novo;
53 }
54
```

```

55 static int tamanho_lista(Node *head) {
56     int n = 0;
57     while (head) { n++; head = head->next; }
58     return n;
59 }
60
61 static void libera_lista(Node *head) {
62     while(head) {
63         Node *tmp = head;
64         head = head->next;
65         free(tmp);
66     }
67 }
68
69 int main(int argc, char *argv[]) {
70     int N = (argc > 1) ? atoi(argv[1]) : 20;
71
72     Node *lista1 = NULL;
73     Node *lista2 = NULL;
74
75     printf("Realizando %d insercoes distribuidas em 2 listas...\n", N);
76
77     #pragma omp parallel
78     {
79         #pragma omp single
80         {
81             for (int i = 0; i < N; i++) {
82                 #pragma omp task firstprivate(i)
83                 {
84                     unsigned int seed = semente_da_thread() ^ (unsigned int)(i *
85                                     2246822519u);
86
87                     int escolha = (int)(xorshift32(&seed) % 2u);
88                     int valor = (int)(xorshift32(&seed) % 1000u);
89
90                     if (escolha == 0) {
91                         #pragma omp critical(lista1_critico)
92                         {
93                             insere_lista(&lista1, valor);
94                         }
95                     } else {
96                         #pragma omp critical(lista2_critico)
97                         {
98                             insere_lista(&lista2, valor);
99                         }
100                     }
101                 }
102             }
103         } /* Fim do Single */
104     }
105
106     printf("Lista 1: %d elementos\n", tamanho_lista(lista1));
107     printf("Lista 2: %d elementos\n", tamanho_lista(lista2));
108
109     libera_lista(lista1);
110     libera_lista(lista2);
111

```

```

112     return 0;
113 }

```

Código 1: Programa com duas listas e regiões críticas nomeadas

A.2 n_listas.c

```

1  /*
2  * n_listas.c
3  *
4  * Compilar: gcc -fopenmp -Wall -O2 n_listas.c -o n_listas
5  * Executar: ./n_listas [M] [N]
6  *           M = número de listas, N = número de inserções
7  */
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <time.h>
12 #include <limits.h>
13 #include <omp.h>
14
15 /* -----
16  * Gerador Xorshift32 (thread-safe: cada chamador guarda seu próprio 'state')
17  * ----- */
18 static inline unsigned int xorshift32(unsigned int *state)
19 {
20     unsigned int x = *state;
21
22     x ^= x << 13;
23     x ^= x >> 17;
24     x ^= x << 5;
25
26     *state = x;
27     return x;
28 }
29
30 /* Geracao de double aleatorio em [0,1) */
31 static inline double random_double(unsigned int *state)
32 {
33     return (double)xorshift32(state) / (double)UINT_MAX;
34 }
35
36 static inline unsigned int semente_da_thread(void)
37 {
38     unsigned int base = (unsigned int)time(NULL) ^
39                         ((unsigned int)omp_get_thread_num() * 2654435761u);
40     return base | 1u;
41 }
42
43
44 typedef struct Node {
45     int data;
46     struct Node *next;
47 } Node;
48
49 /* Insere no inicio da fila -- Não éthread-safe pro si só*/
50 static void insere_lista(Node **head, int valor) {

```

```

51     Node *novo = malloc(sizeof(Node));
52     novo->data = valor;
53     novo->next = *head;
54     *head = novo;
55 }
56
57 static int tamanho_lista(Node *head) {
58     int n = 0;
59     while (head) { n++; head = head->next; }
60     return n;
61 }
62
63 static void libera_lista(Node *head) {
64     while(head) {
65         Node *tmp = head;
66         head = head->next;
67         free(tmp);
68     }
69 }
70
71 int main(int argc, char *argv[]) {
72     int M = (argc > 1) ? atoi(argv[1]) : 4;
73     int N = (argc > 2) ? atoi(argv[2]) : 40;
74
75     if (M <= 0) {
76         fprintf(stderr, "M precisa ser >= 1\n");
77         return 1;
78     }
79
80     Node **listas = calloc((size_t)M, sizeof(Node *));
81     omp_lock_t *locks = malloc((size_t)M * sizeof(omp_lock_t));
82
83     for (int i = 0; i < M; i++) {
84         omp_init_lock(&locks[i]);
85     }
86
87     printf("Realizando %d insercoes distrubuidas em %d listas...\n", N, M);
88
89     #pragma omp parallel
90     {
91         #pragma omp single
92         {
93             for (int i = 0; i < N; i++) {
94                 #pragma omp task firstprivate(i)
95                 {
96                     unsigned int seed = semente_da_thread() ^ (unsigned int)(i *
97                                     2246822519u);
98
99                     int idx = (int)(xorshift32(&seed) % (unsigned int)M); /* lista
100                                     escolhida */
101                     int valor = (int)(xorshift32(&seed) % 1000u);
102
103                     omp_set_lock(&locks[idx]);
104                     insere_lista(&listas[idx], valor);
105                     omp_unset_lock(&locks[idx]);
106                 }
107             }
108         }
109     }
110 }

```

```

107     }
108
109     for (int i = 0; i < M; i++) {
110         printf("Lista %d: %d elementos\n", i, tamanho_lista(listas[i]));
111         libera_lista(listas[i]);
112     }
113
114     for (int i = 0; i < M; i++) {
115         omp_destroy_lock(&locks[i]);
116     }
117
118     free(locks);
119     free(listas);
120
121     return 0;
122 }

```

Código 2: Programa generalizado para M listas com locks explícitos